SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr. Hemavathi R

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and	BL4 – Analyze	**5**

	secondary indexing to support fast queries by department and CGPA range.		
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I Sree B (192524023) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Sree B

1. Question

An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Answer

For an e-commerce company with **10 million product records**, a **B+ Tree-based indexed sequential organization** is recommended.

Proposed Design

- Store product records in a data file.
- Use Product_ID as the primary key.
- Create a **B+ Tree primary index** on Product_ID.
- Create secondary indexes on frequently searched attributes such as Category_ID and Brand_ID.
- Keep records physically organized according to the primary key where appropriate.

Cost Analysis

Assume:

- Number of records = 10,000,000
- Record size = 200 bytes
- Block size = 4 KB = 4,096 bytes

Records per block:

Number of data blocks:

Thus, approximately **500,000 data blocks** are required.

A B+ Tree has high fan-out, so the number of index levels can remain small even for millions of records.

Advantages

1. Fast primary-key searches.
2. Efficient range searches.
3. Good sequential access.
4. Suitable for large datasets.
5. B+ Tree indexes can grow dynamically.
6. Secondary indexes support searches by category and brand.

Conclusion

For 10 million products, **B+ Tree indexed organization** provides a good balance between storage cost, search performance, and scalability. Although indexes consume additional storage, they significantly reduce disk I/O during searches.

2. Question

A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Answer

A banking system needs both **exact-match** and **range** searches.
Suppose the transaction table is:

```
Transaction(
   Transaction_ID,
   Account_No,
   Transaction_Date,
   Amount,
   Transaction_Type
)
```

Recommended Indexes

1. Composite B+ Tree Index

Create:
(Account_No, Transaction_Date)
This index is suitable for queries such as:

```
SELECT *
FROM Transaction
WHERE Account_No = 1001
AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';
```

The first column provides account-based filtering, while the second supports the date range.

2. Primary Index

Use:
Transaction_ID
as the primary key.

3. Optional Secondary Index

If transactions are frequently searched by date alone, create:
(Transaction_Date)

Analysis

Requirement	Recommended Index
Transaction ID	Primary B+ Tree
Account number	B+ Tree
Account + date range	Composite B+ Tree
Date-only queries	Secondary B+ Tree

Conclusion

The **composite B+ Tree index on (Account_No, Transaction_Date)** is the most important index because it efficiently supports both account-number searches and date-range queries.

3. Question

A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Answer

Suppose the patient table is:

```
Patient(
   PatientID,
   PatientName,
   DoctorName,
   Age,
   Disease
)
```

Two major search requirements are:

1. Search by PatientID.
2. Search by DoctorName.

Proposed Multi-Level Index

Level 1 – Primary Index

Create a B+ Tree on:

PatientID

The PatientID index provides fast access to individual patient records.

Level 2 – Secondary Index

Create a B+ Tree on:

DoctorName

This allows the system to find all patients belonging to a particular doctor.

Structure

```
Primary B+ Tree
      |
 PatientID
      |
 Patient Records

Secondary B+ Tree
      |
  DoctorName
      |
 Patient Records
```

For a very large database, the indexes themselves can be organized as multiple levels:

```
Level 1
  ↓
Level 2
  ↓
Leaf-level index
  ↓
Patient Records
```

Advantages

- Fast PatientID lookup.
- Efficient doctor-based retrieval.
- Reduced disk I/O.
- Multi-level indexing prevents the index from becoming excessively large.
- B+ Tree leaves support sequential access.

Conclusion

Use a **primary multi-level B+ Tree index on PatientID** and a **secondary multi-level B+ Tree index on DoctorName**.

4. Question

A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Answer

For a logistics company, shipment records may be continuously inserted, updated, and deleted. Both B-Trees and B+ Trees support dynamic operations, but **B+ Tree is generally more suitable**.

Comparison

Feature	B-Tree	B+ Tree
Insertion	Efficient	Efficient
Deletion	Efficient	Efficient
Search	Efficient	Efficient
Range search	Good	Excellent
Sequential access	Good	Excellent
Leaf linking	Usually absent	Present
Database suitability	Good	Excellent

Why B+ Tree?

Shipment systems may require queries such as:
Find shipments from 1 August to 15 August.
B+ Tree leaf nodes are linked:
[1-5] → [6-10] → [11-15] → [16-20]
Therefore, after reaching the first matching leaf, the system can scan neighboring leaves efficiently.

Conclusion

B+ Tree is recommended because it provides:

- Efficient dynamic insertion.
- Efficient deletion.
- Fast exact searches.
- Excellent range queries.
- Efficient sequential access.
- Good scalability for large databases.

5. Question

Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Answer

Suppose each employee has:
Employee_ID
Employee_Name
Department
Salary
The primary access requirement is employee lookup by Employee_ID.

Hash Function

A simple hash function can be:
For example, with 1000 buckets:
h(5012) = 5012 % 1000 = 12

Static Hashing

In static hashing, the number of buckets is fixed.
For example:
1000 buckets
5000 employees
Average records per bucket:

Advantages:

- Simple implementation.
- Low management overhead.
- Fast exact-match search.

Disadvantages:

- Difficult to expand.
- Overflow chains can occur.
- Performance may decrease as the database grows.

Extendible Hashing

Extendible hashing dynamically splits buckets when they overflow.

Advantages:

- Supports database growth.
- Reduces long overflow chains.
- Good search performance.
- Automatically expands the directory.

Disadvantages:

- More complex.
- Directory requires additional memory.

Comparison

Feature	Static Hashing	Extendible Hashing
Implementation	Simple	More complex
Growth	Poor	Excellent
Bucket splitting	No/limited	Yes
Overflow	Overflow chains	Bucket splitting
Scalability	Moderate	High

Conclusion

For exactly **5000 relatively stable employees**, static hashing is sufficient. However, for an HR system expected to grow significantly, **extendible hashing is the better long-term solution**.

6. Question

An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Answer

The airline system has two different query types:

1. Exact search by Flight_Number.
2. Range search by Flight_Date.

Recommended Design

Use two indexes:

1. Hash Index on Flight Number

Hash(Flight_Number)

This provides fast exact-match queries.

Example:

```
SELECT *
FROM Flight
WHERE Flight_Number = 'AI202';
```

Hashing can directly identify the bucket containing the flight.

2. B+ Tree on Flight Date

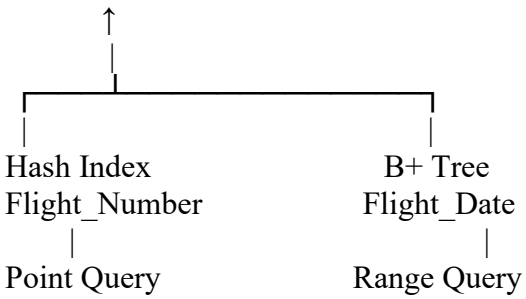
Create:

B+ Tree(Flight_Date)

This supports:

```
SELECT *
FROM Flight
WHERE Flight_Date BETWEEN '2026-08-01'
AND '2026-08-15';
```

Combined Structure



Conclusion

Use a **hash index for Flight_Number** and a **B+ Tree index for Flight_Date**. This combination provides efficient performance for both exact-match and range queries.

7. Question

For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Answer

Suppose the database contains:

```
Student(
   Student_ID,
   Name,
   Department,
   CGPA,
   Year
)
```

File Organization

A **heap file organization** can be used if student records are frequently inserted and updated.

The primary key is:

Student_ID

A primary B+ Tree can be maintained on Student_ID.

Secondary Index 1 – Department

Create a B+ Tree secondary index on:

Department

This allows queries such as:

```
SELECT *
FROM Student
WHERE Department = 'CSE';
```

Secondary Index 2 – CGPA

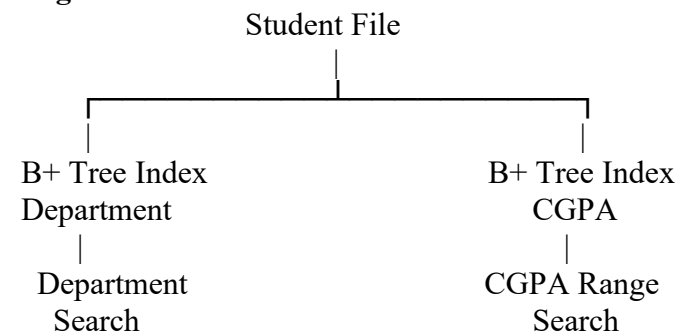
Create a B+ Tree secondary index on:

CGPA

This efficiently supports:

```
SELECT *
FROM Student
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

Design



Conclusion

Use **heap organization for flexible record insertion**, a primary index on Student_ID, and secondary B+ Tree indexes on Department and CGPA. This provides efficient department and CGPA-range queries.

8. Question

Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Answer

Given:

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Step 1: Calculate Records per Block

Therefore, each block can store **81 records**.

Step 2: Calculate Number of Blocks

Step 3: Utilization

Records occupy:

Allocated space:

Approximate utilization:

Comparison

Organization	Approx. Data Blocks	Main Characteristic
Heap	1,235	Unordered records
Sorted	1,235 + possible overhead	Records maintained in sorted order
Hashed	≥1,235 + possible overflow space	Records distributed among buckets

Analysis

Heap:

Provides simple insertion and good space utilization, but searches may require scanning many blocks.

Sorted:

Provides excellent ordered and range retrieval, but insertion and deletion are more expensive.

Hashed:

Provides fast exact-match retrieval but may require additional overflow blocks when collisions occur.

Conclusion

The basic data requirement is approximately **1,235 blocks** for all three organizations. However, actual hashed and indexed organizations may require additional blocks for overflow and index structures.

9. Question

A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Answer

Suppose the post table contains:

```
Post(
   Post_ID,
   User_ID,
   Timestamp,
   Hashtag,
   Content
)
```

With **500 million records**, a single index is not sufficient for all query types.

1. User-Based Queries

Create a composite B+ Tree:

(User_ID, Timestamp)

This supports:

```
SELECT *
FROM Post
WHERE User_ID = 1001
ORDER BY Timestamp;
```

It also efficiently retrieves a user's posts within a time range.

2. Hashtag Queries

Create a secondary index:

(Hashtag, Timestamp)

This supports:

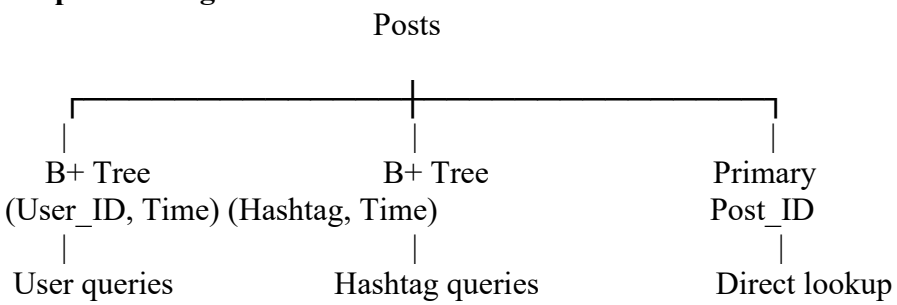
```
SELECT *
FROM Post
WHERE Hashtag = '#database'
AND Timestamp BETWEEN ...;
```

3. Post ID

Create a primary index:

Post_ID

Proposed Design



Important Consideration

Because the database contains **500 million posts**, indexes can become very large. Therefore:

- Partition data by time.
- Use appropriate composite indexes.
- Avoid unnecessary indexes.
- Consider distributed/sharded storage.
- Periodically archive old posts if required.

Conclusion

A combination of **(User_ID, Timestamp) and (Hashtag, Timestamp) B+ Tree indexes**, together with a primary Post_ID index, provides efficient access for the major query patterns while allowing the system to scale.

10. Question
Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Answer
For a supermarket billing system, transactions are frequently inserted and may be searched using transaction ID, customer ID, or other attributes.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	Very fast	Slow	Very fast
Delete	Fast	Slow/moderate	Fast
Exact search	Slow	Fast with binary/index search	Very fast
Range search	Slow	Excellent	Poor
Sequential scan	Good	Excellent	Moderate
Maintenance	Simple	More complex	Moderate

Heap File
New transactions can simply be appended.

Insertion: Very fast.
However, finding a particular transaction may require a sequential scan.

Sorted File
Transactions are maintained in sorted order.

Advantages:
- Excellent binary search.
- Excellent range queries.

Disadvantage:
- Inserting 1 million transactions continuously can require expensive reorganization or additional management.

Hashed File
Use:
h(Transaction_ID) = Transaction_ID % number_of_buckets
This provides very fast exact-match retrieval.
For example:
SELECT *
FROM Transaction

WHERE Transaction_ID = 500123;
The hash function can directly identify the relevant bucket.
However, hashing is not suitable for queries such as:
WHERE Transaction_ID BETWEEN 500000 AND 501000

Overall Evaluation

Requirement	**Best Choice**
Very frequent insertion	Heap
Exact transaction lookup	Hash
Range queries	Sorted
Sequential processing	Sorted/Heap
Large-scale exact-match transactions	Hash

Recommendation
For a supermarket billing system with **1 million daily transactions**, a **hybrid approach** is recommended:

- Use efficient append-oriented storage for transaction ingestion.
- Use a **hash index** for exact transaction-ID lookup.
- Use a **B+ Tree index** for date/time and range-based queries.
- Partition or archive transactions based on date to control the size of active data.

Conclusion: There is no single organization that is optimal for every operation. Heap files provide excellent insertion performance, sorted files provide excellent ordered and range retrieval, and hashed files provide excellent exact-match retrieval. A combination of these techniques is the most appropriate design for a high-volume billing system.